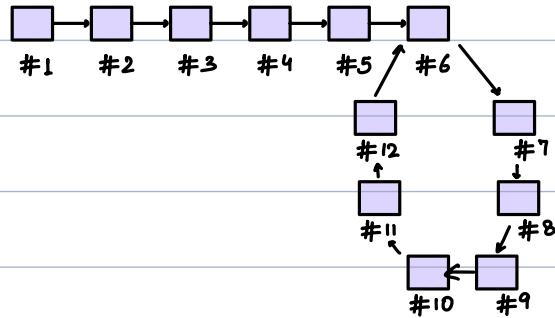
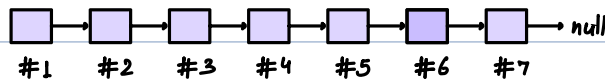


Linked List Cycle Detection: Given a head node of a LL, check for the cycle



true



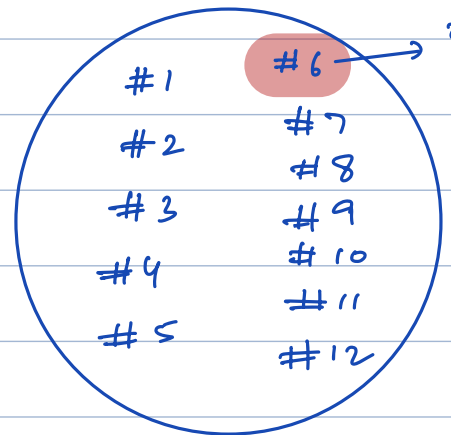
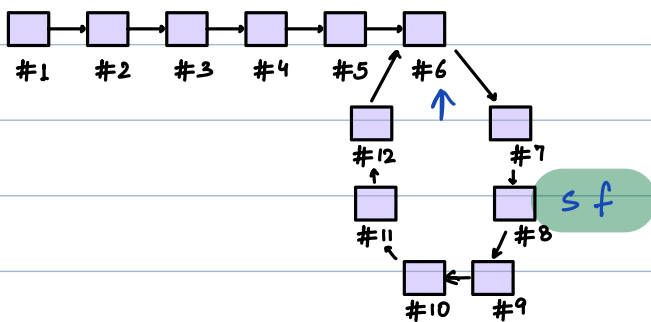
false

Idea 1:

Use a hashset

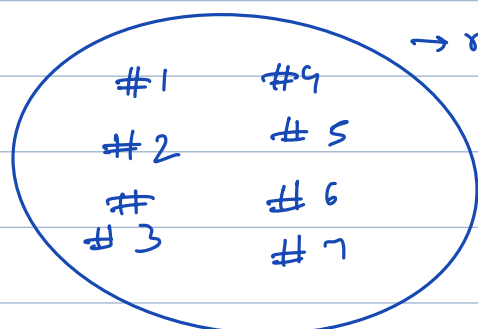
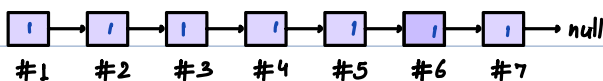
HashSet < Node > hs = new HashSet <>();

Ex



return true
cycle exists

Ex



→ return false
no cycle

Pseudocode:

```
HashSet <Node> hs = new HashSet<>();
```

```
cur = head
```

```
if (cur == null)
```

```
    return false
```

```
while (cur != null) {
```

```
    if (hs.contains(cur)) {
```

```
        return true
```

```
    }
```

```
    else {
```

```
        hs.add(cur); cur = cur.next;
```

```
    }
```

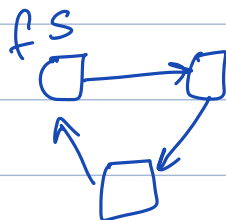
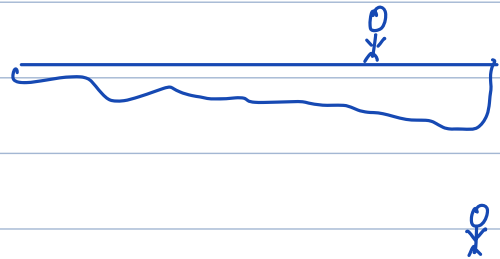
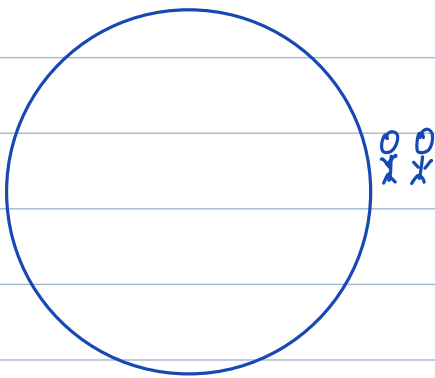
```
}
```

TC: $O(N)$

SC: $O(N)$

Idea 2:

slow pointer and fast pointer



```
boolean detectCycle(Node h){
```

```
    Node slow = h;
```

```
    Node fast = h;
```

```
    // if linked list is empty
```

```
    if (h == null)
```

```
        return false;
```

```
    boolean isCycle = false
```

```
    while (fast.next != null && fast.next.next != null) {
```

```
        slow = slow.next;
```

```
        fast = fast.next.next;
```

```
        if (slow == fast) { isCycle = true; break; }
```

```
    }
```

```
    return isCycle;
```

```
}
```

TC: $O(N)$

SC: $O(1)$